# Separating Statistical Inference from Experimental Runtime: An Architecture for Sequential Scientific Analysis

Alexander Stepanov
Sobolev Institute of Mathematics

Andrej Novikov
Novikov Laboratories LLC

**Abstract**

Sequential statistical procedures are increasingly embedded in scientific systems in which observations arrive incrementally from heterogeneous sources. In such systems the statistical update is only one part of the computation: observations must also be acquired, aligned across inputs, ordered, delivered, persisted, and replayed, and results must be passed to systems that display them or act on them. Sequential methods are nevertheless missing from the general software toolkit. They run inside proprietary experimentation platforms and commercial statistical packages, or exist as offline functions over complete data, and the batch `fit` interface that dominates machine-learning libraries has no place for a procedure that listens, keeps state, and decides when to stop.

We propose an architecture that separates statistical inference from data topology, execution runtime, run lifecycle, persistence, and external action. Its central abstraction is the *Sequential Procedure*: a state-transition function that maps an inferential state, a valid statistical input, and a statistical random state to a new state and a scientific output. The procedure does not define how observations are acquired, paired, scheduled, stored, or acted upon. We specify the architecture semantically rather than through an object model, derive seven conformance invariants that any implementation can be tested against (among them synchronous/asynchronous, checkpoint, replay, and batch equivalence), and describe a dependency-free reference implementation in which all seven hold bitwise. Lorden's 2-SPRT for the Kiefer–Weiss problem serves as the reference procedure; the same contract accommodates sequential estimators, Bayesian filters, change-point detectors, and stateful predictive models.

**Keywords:** sequential inference, sequential statistics, scientific computing, streaming data, experimental runtime, checkpointing, provenance, data topology, Kiefer–Weiss, stateful inference

## 1 Introduction

Many scientific analyses are inherently sequential. Observations are collected over time, the available information changes with every observation, and the current inferential state may determine what is measured next or whether measurement continues at all. Sequential hypothesis testing [1, 2], change-point detection [3, 4], Bayesian filtering and state-space inference [5, 6], and sequential prediction are all of this kind.

The mathematical definition of such a procedure is usually small: given a state and a new observation, produce a new state and an output. A working scientific implementation has to solve a much larger problem. Observations originate from heterogeneous sources, arrive asynchronously, need to be aligned with one another, may be delivered in batches, duplicated, or reordered, and have to be persisted so that an experiment that runs for days can be interrupted and resumed.

**Fit is not listen.** The interface that dominates machine-learning software does not have this shape. A `fit` method receives a complete data set and returns a fitted object [7]. A sequential procedure has to *listen*: it receives one observation, updates a state, and may announce that it has seen enough. Incremental variants such as `partial_fit` process data in chunks, but they treat samples as exchangeable. They carry no step index, no stopping rule, and no state that means anything as a point in a sequence. Even models that are sequential by construction are commonly used through the batch interface. A widely used hidden Markov model library offers estimation, decoding, and scoring of whole sequences [8]; the forward recursion that updates a belief after each observation is not part of its public interface. Recurrent networks are often trained and applied on fixed windows, with the hidden state reset between windows. In both cases the state that makes the model sequential is discarded at the boundary of every call.

**Where sequential methods live today.** Sequential methods with decision semantics are in production use, yet they are outside the general toolkit (Table 1). Commercial experimentation platforms run mixture SPRTs and anytime-valid confidence sequences on live data [9, 10, 11, 12]. Commercial statistical software and a validated R package provide group-sequential designs for clinical trials [13, 14, 15]. The former are proprietary and embedded in a platform. The latter operate on assembled data sets at interim analyses. Open implementations of fully sequential tests are mostly R functions that take the complete vector of observations and return a decision [16, 17, 18]. That is of limited use in application engineering. R is an environment for analysis and is rarely the language in which a deployed system is written: in the 2025 Stack Overflow survey, 3.3% of professional developers reported extensive development work in R, against 54.8% for Python [19]. Using an R function from a service means operating a second runtime behind a bridge, or re-implementing the method, and that cost is paid again for every method adopted. Open incremental libraries exist for change detection and drift alarms [20, 21]; their state is mutable and internal, and the stopping rule is left to the caller. We found no packaged implementation of Lorden's 2-SPRT. The available code for the Kiefer–Weiss problem, including our own, computes optimal designs offline [22, 23, 24].

The gap is therefore not in theory and not in demand. What is missing is a generic, open component model for sequential procedures, as opposed to one more implementation of one method inside one system. Numerical computing was once in the same position. The algorithms existed as Fortran routines, and what general-purpose libraries such as SciPy [25] added was a uniform and accessible interface to them, with no new numerical methods. That step is what turned specialist routines into a general toolkit, and sequential inference has not yet taken it.

When such methods are implemented ad hoc, the surrounding concerns end up inside the statistical method. The implementation of a test or a filter then depends on a particular event loop, message broker, database, sensor API, file format, or plotting library. None of these belongs to the mathematical definition of the procedure, yet they become inseparable from its software representation. The consequences are practical. The method cannot be reused in another laboratory without its infrastructure, it cannot be validated in isolation, and a run cannot be reproduced without reproducing the environment that happened to execute it.

This paper proposes a separation, stated as a single principle:

> A sequential statistical procedure should define how valid observations update inferential state and produce scientific outputs. It should not define how observations are acquired, scheduled, transported, persisted, or acted upon.

Two aspects of this separation deserve emphasis because they are where existing designs most often leak.

The first is *topology*. A sequential scientific system is not a single stream-processing function. A procedure may receive two independent samples, paired observations, or several heterogeneous inputs with different acquisition mechanisms and different clocks. Whether $A_3$ is analyzed together with $B_3$, with the $B$ nearest in time, or on its own is a property of the experimental design. It is not a property of the likelihood update, and it should not be reconstructed inside it.

The second is *action*. A procedure produces scientific outputs; other components may use those outputs for visualization, logging, experimental design, alerting, or control. We summarize this as

**Procedure infers; consumers act.**

**Contributions.** The paper makes four contributions.

(i) A semantic definition of a Sequential Procedure as a state-transition function with an explicit statistical random state, partial multi-input structure, and absorbing terminal states (Section 5).

(ii) A layered architecture around that definition, with precise responsibilities for adapters, input contracts, data topology, runtime, run, and consumers (Sections 4, 6 to 8 and 10).

(iii) Seven conformance invariants that follow from the architectural principles and can be checked mechanically for any implementation (Section 13).

(iv) A reference implementation with no dependencies in which the invariants are executable tests, and a worked reference procedure for the Kiefer–Weiss problem (Sections 12 and 14).

## 2 Background and Related Work

**Sequential analysis.** The statistical side of the problem is classical. Wald's sequential probability ratio test (SPRT) [1, 2] and its optimality [26], Page's cumulative-sum scheme [3], and the theory built on them [27, 4, 28] define procedures by a recursion on a low-dimensional statistic and a stopping rule. The more recent literature on anytime-valid inference [29, 30] does the same with e-processes and confidence sequences. Filtering and state-space methods [5, 31, 32, 6, 33, 34] and recurrent predictive models [35] share that recursive shape. In all of these the published object is a transition rule. What the literature does not prescribe, reasonably, is how the rule should be embedded in a running system.

**Software for sequential inference.** Table 1 summarizes where sequential inference is available as software. The table was compiled in October 2026 from package documentation and published descriptions. It is not exhaustive, and for proprietary systems it reflects only what their authors have published.

Three groups can be distinguished. The first has decision semantics and is production-grade, but it is proprietary, commercially licensed, or tied to interim analyses of assembled data. The mixture SPRT of Johari et al. [9, 10] and the staged-rollout tests of Zhao et al. [11] are examples of the former; group-sequential software for clinical trials [13, 14, 15] is an example of the latter. SAS's analysis procedure is notable because it passes a boundary data set from one interim stage to the next, which is a documented form of resumable state at the granularity of stages. The second group is open and has decision semantics, but consumes complete data vectors and keeps nothing between calls [16, 17, 18]. The third group is open and incremental, but leaves the state implicit and the decision to the caller: boundary functions for confidence sequences [30], online change-point detectors [20], drift detectors [21], and a streaming two-sample test that reports fixed-horizon $p$-values without a stopping rule [37].

Table 1: Where sequential inference is available as software. "Consumes" is the unit of data handed to the method in one call. "State" is what the caller can keep between calls in order to continue.

| System | Access | Consumes | State | Stopping rule |
|---|---|---|---|---|
| Optimizely, Statsig [9, 11] | proprietary platform | live metrics | not public | yes |
| Netflix [12] | internal | live counts | not public | yes |
| SAS SEQDESIGN, SEQTEST [13] | commercial | interim stage | boundary data set | yes |
| rpact [14] | open, R | interim stage | R objects | yes |
| sprtt, MSPRT [16, 17] | open, R | full vector | none | yes |
| safestats [18] | open, R | full vector | none | yes |
| confseq [30] | open, Python | running statistics | held by caller | left to caller |
| Confidence [36] | open, Python | data frame per look | none | bounds only |
| focus, focus-cpt [20] | open, R and Python | one observation | mutable, internal | left to caller |
| River drift detectors [21] | open, Python | one observation | mutable, internal | alarm flag |
| Spark `StreamingTest` [37] | open, JVM | micro-batch | not exposed | none |
| `partial_fit` [7] | open, Python | mini-batch | model parameters | none |
| hmmlearn [8] | open, Python | whole sequence | none | — |
| Kiefer–Weiss code [22] | open, R scripts | design parameters | — | plan only |
| This work | open, Python | one statistical input | explicit, serializable, with random state | terminal state |

None of the systems we examined offers a sequential test as a component that consumes one observation at a time, exposes a terminal decision, and keeps its inferential and random state in a form that can be persisted and resumed. For the Kiefer–Weiss problem, earlier work co-authored by one of us [22, 24] and its companion [23] compute optimal tests by backward induction. The accompanying code produces the plan, a table of continuation intervals, and does not execute it on a stream (Section 12).

**Incremental learning interfaces.** Libraries for online machine learning expose incremental update methods on mutable model objects [38, 7, 21, 39]. This is close in spirit to a Sequential Procedure. The differences lie in what is made explicit: these interfaces leave ordering, persistence of randomness, multi-input topology, stopping, and the provenance of a run to the caller.

**Stream processing.** Distributed stream processors solved many of the execution problems we discuss: event time versus processing time, windowing, out-of-order data, and consistent snapshots of operator state [40, 41, 42, 43, 44, 45]. Their determinacy results go back to Kahn's process networks [46]. Summingbird [47] showed that pure aggregators can be executed unchanged by a batch and a streaming runtime, which is a direct precedent for batch transparency. It relies on associativity, which a procedure with a stopping rule does not have. These systems treat operator state as opaque and optimize for throughput and fault tolerance

at scale. They do not say what a statistical random state is, when two state trajectories count as equal, or what a terminal decision means for acquisition. Our architecture borrows their vocabulary and is meant to be hosted by such systems as readily as by a for-loop; it specifies the part they leave open.

**Experiment orchestration.** Frameworks for instrument control separate the description of an experiment from the engine that executes it and from the consumers of its data [48]. This is the closest precedent for our separation of acquisition and consumers. Such frameworks do not define a contract for the statistical procedure that sits between the two.

**Reproducibility and provenance.** Guidelines for reproducible computational research [49, 50] and provenance models [51] ask that the lineage of a result be recorded. For sequential experiments the lineage includes things that batch analyses do not have: the order in which observations were applied, what was dropped under load, and the random state at every interruption.

**Software architecture and novelty.** The separation itself is an instance of information hiding [52] and of the ports-and-adapters style [53]. Formally, a Sequential Procedure is a Mealy machine [54] with a possibly infinite state space. Neither the automaton nor the idea of separating pure logic from runtime and persistence is new. What we add is the application of that pattern to sequential procedures with decision semantics, and the parts it then requires: a statistical random state that is carried in the contract, a topology layer that relates multiple inputs before inference, and conformance invariants against which any implementation can be tested. As with general-purpose scientific libraries [25], the value of a common contract does not depend on the methods behind it being new. Existing kernels, including those written in R or C++, can be placed behind it without being rewritten.

# 3 Architectural Principles

The architecture rests on seven principles. Each is used later to derive a testable invariant (Section 13).

**Principle 1** (Inference–execution separation)**.** The state transition of a procedure is independent of the mechanism that delivers its inputs.

**Principle 2** (Topology–inference separation)**.** Joining, pairing, alignment, filtering, mapping, and windowing of data sources occur outside the procedure.

**Principle 3** (Logical inputs)**.** A procedure consumes logical statistical inputs, not physical sources. Physical acquisition is connected through adapters.

**Principle 4** (Single-writer inference)**.** For one procedure instance, state transitions are serialized according to an explicit ordering policy.

**Principle 5** (Persistent state)**.** The state required to continue a run is distinct from, and independent of, the disposable execution state of the runtime.

**Principle 6** (Consumer non-interference)**.** Consumers observe outputs. They cannot modify inferential state, and their failures are not inference failures.

**Principle 7** (Semantic portability)**.** The architecture is defined by semantic contracts, not by a particular programming-language object model.

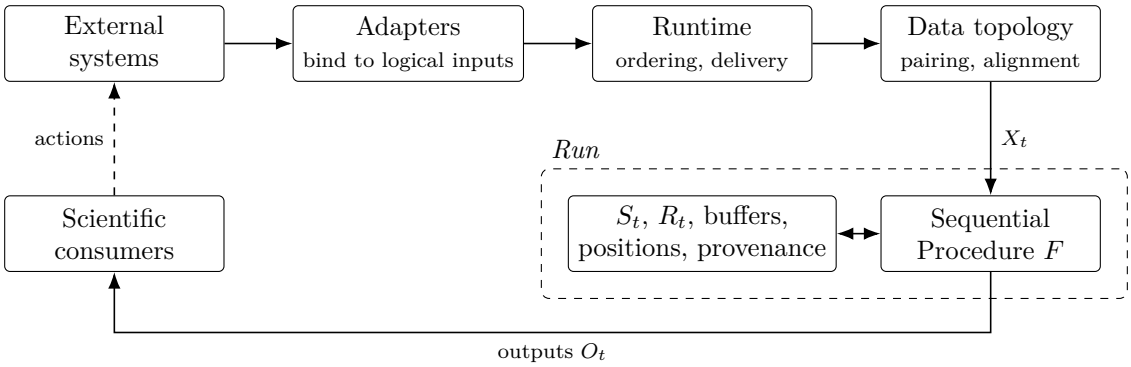



Figure 1: The data path. The run is not a stage on the path: it is the context that owns everything which must outlive the executing process. Actions return to the external world, never directly to the procedure.

# 4 System Overview

Figure 1 shows the data path. Observations leave an external system, are bound to logical inputs by an adapter, are ordered and deduplicated per source by the runtime, and are related to one another by the data topology. What crosses into the run is a *statistical input* $X_t$. The procedure maps it to an output $O_t$, which is handed to consumers.

The layers have the following responsibilities.

**Adapters**
: connect external systems to logical inputs. They convert formats and units and attach source identity, sequence numbers, keys, and event times.

**Logical inputs**
: carry a contract that states what a valid observation is (Section 6.1).

**Runtime**
: decides how observations are acquired and scheduled, and in which per-source order they are delivered (Section 7). It holds no state that needs to survive.

**Data topology**
: relates inputs to one another and emits statistical inputs (Section 6.2).

**Run**
: is one concrete execution. It owns the inferential state, the statistical random state, topology buffers, source positions, lifecycle, and provenance, and it is the single writer of the inferential state (Section 8).

**Sequential Procedure**
: performs the state transition (Section 5).

**Scientific consumers**
: use outputs for observation, reporting, design, or action (Section 10).

Per-source ordering precedes topology because positional pairing is only meaningful on ordered inputs. Serialization of state transitions follows topology because that is where statistical inputs come into existence.

Not every application instantiates every layer. A minimal offline analysis reduces to a procedure folded over an iterator. A laboratory system instantiates all of them. The procedure is the same object in both.

Table 2: Examples of inferential state.

| **Procedure** | **State** $S_t$ | **Terminal** |
|---|---|---|
| Likelihood-ratio test | $(t,\ \log \Lambda_t,\ d_t)$ | decision $d_t$ taken |
| CUSUM detector | $(t,\ W_t)$ | alarm raised |
| Kalman filter | $(\hat{z}_t,\ P_t)$ | never |
| Bayesian filter | $p(z_t \mid x_{1:t})$ or a particle cloud | never |
| LSTM predictor | $(h_t,\ c_t)$ | never |

# 5 Sequential Procedure Semantics

**Definition 1** (Sequential Procedure)**.** A sequential procedure is a tuple $(\mathcal{S}, \mathcal{X}, \mathcal{O}, \mathcal{R}, s_0, F, \mathcal{S}_\top)$ consisting of a state space $\mathcal{S}$, an input space $\mathcal{X}$, an output space $\mathcal{O}$, a space $\mathcal{R}$ of statistical random states, an initial state $s_0 \in \mathcal{S}$, a transition function

$$F : \mathcal{S} \times \mathcal{X} \times \mathcal{R} \to \mathcal{S} \times \mathcal{O} \times \mathcal{R},$$

and a set $\mathcal{S}_\top \subseteq \mathcal{S}$ of terminal states.

Given an initial state $S_0 = s_0$, an initial random state $R_0$, and statistical inputs $X_1, X_2, \ldots$, the procedure evolves as

$$(S_t,\, O_t,\, R_t) \;=\; F(S_{t-1},\, X_t,\, R_{t-1}), \qquad t = 1, 2, \ldots \tag{1}$$

so that $S_t$ and $O_t$ are functions of $X_{1:t}$. For a deterministic procedure $\mathcal{R}$ is a single point and (1) reduces to $(S_t, O_t) = f(S_{t-1}, X_t)$.

**Definition 2** (Trajectory)**.** For an input history $x_{1:T}$, the trajectory is

$$\mathcal{T}(x_{1:T}; s_0, r_0) \;=\; \big((S_t, O_t, R_t)\big)_{t=1}^{\tau}, \qquad \tau = \min\big(T,\ \inf\{t : S_t \in \mathcal{S}_\top\}\big).$$

The trajectory is the reference semantics of a run. Every claim of the form "mechanism $A$ is equivalent to mechanism $B$" in this paper means that both produce the same trajectory, under a stated notion of equality (Section 9).

The definition is deliberately broader than hypothesis testing. The state may be a test statistic, an estimator, a posterior distribution, a latent-state belief, or the hidden state of a predictive model; Table 2 gives examples. The architecture imposes no universal representation on $\mathcal{S}$. It requires only that a state can be encoded to a value and decoded again without loss.

## 5.1 Statistical inputs

Let a procedure declare $m$ logical inputs with value spaces $\mathcal{X}^{(1)}, \ldots, \mathcal{X}^{(m)}$. Requiring every step to carry all of them would exclude the most common two-sample situation, in which observations of independent samples arrive one at a time and in no fixed relation. We therefore define a statistical input as a partial assignment

$$X_t \;=\; \big(X_t^{(j)}\big)_{j \in J_t}, \qquad \emptyset \neq J_t \subseteq \{1, \ldots, m\}. \tag{2}$$

A procedure is *joint* if it requires $J_t = \{1, \ldots, m\}$ at every step, as a paired test does. Otherwise it accepts any non-empty subset. The inputs stay logically distinct in either case (Figure 2).

This avoids reducing the system to one undifferentiated event stream. A single stream would leave the relation between inputs implicit and force the procedure to reconstruct it, which is exactly the coupling Principle 2 rules out.

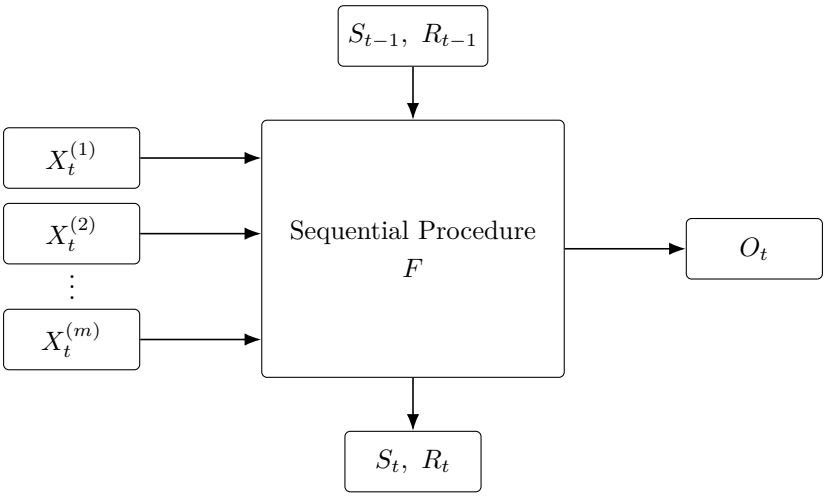



Figure 2: A procedure consumes logically distinct statistical inputs, any non-empty subset of which may be present at a step. It receives statistical inputs, not physical streams.

### 5.2 Sample structure and observation dimensionality

The number of samples and the dimensionality of an observation are independent axes. A one-sample multivariate procedure has $X_t \in \mathbb{R}^d$; a two-sample scalar procedure has $X_t^{(1)}, X_t^{(2)} \in \mathbb{R}$; a two-sample multivariate procedure has $X_t^{(1)}, X_t^{(2)} \in \mathbb{R}^d$. Sample structure, observation structure, and alignment are therefore expressed as contracts and metadata. The architecture needs no separate type hierarchies for one-, two-, and $k$-sample procedures.

### 5.3 Terminal states

Terminal states are absorbing: once $S_t \in \mathcal{S}_\top$, no further transition is applied and later statistical inputs do not alter the trajectory. Making this part of the definition matters for two reasons. It fixes the meaning of "the same history" when a decision falls in the middle of a batch (Section 7.2). And it separates the statistical event (a decision has been taken) from the operational one (acquisition stops), which the runtime may schedule later (Section 8.2).

### 5.4 Randomized procedures

Three kinds of randomness occur in a sequential experiment and must not share a generator.

1. *Statistical randomness* is drawn by $F$ itself, as in a particle filter or a randomized test. Its state $R_t$ is part of the trajectory.
2. *Simulation randomness* generates synthetic observations. It belongs to a source.
3. *Execution randomness* arises from scheduling, retries, or jitter. It belongs to the runtime and must not influence the trajectory.

Because $R_t$ appears in (1), reproducing a run that was interrupted at step $k$ requires $R_k$. The initial seed is not enough unless the whole history is replayed. A checkpoint therefore stores the current random state, and the transition commits $S_t$ and $R_t$ together. For a fixed implementation and numerical contract, $(x_{1:T}, s_0, r_0)$ determines $\mathcal{T}(x_{1:T}; s_0, r_0)$.

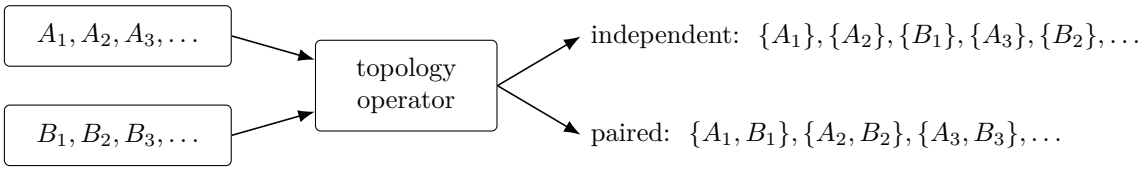



Figure 3: The same two sources under two topologies. Pairing and alignment happen before statistical inference, not inside it.

# 6 Inputs: Contracts and Topology

## 6.1 Input contracts

Every logical input $j$ carries a contract $\mathsf{IC}_j$. A contract specifies the name and semantic identity of the input, its data type and shape, its domain, its unit, its missing-value policy, and any metadata required for ordering, alignment, or provenance. A value is valid if

$$x \models \mathsf{IC}_j. \tag{3}$$

Validation happens before the transition. An input that fails its contract never reaches $F$, so a contract violation cannot leave a partially updated state. Because the contract belongs to the logical input, a physical source can be replaced by any other source that satisfies the same contract without touching the procedure (Principle 3).

## 6.2 Data topology

Data topology specifies how logical inputs are related before they cross the statistical boundary. Consider two sources $A$ and $B$ that produce $A_1, A_2, \ldots$ and $B_1, B_2, \ldots$ respectively. For independent samples each observation is a statistical input of its own. For a paired analysis a topology operator constructs the pairs (Figure 3). The two analyses use the same sources and different topologies. A paired test, moreover, needs no procedure of its own: it is a one-sample test preceded by pairing and a difference map.

Topology operations include positional pairing, key-based joins, timestamp alignment with a tolerance, windowing, filtering, and stateless mapping. Two properties of these operators have consequences for the rest of the architecture.

**Topology operators have state.** A pairing operator that has received $A_4$ but not yet $B_4$ holds an observation that has been consumed from its source and has not reached the procedure. If the run is interrupted at that moment, $A_4$ exists only in the operator's buffer. Topology state is therefore part of what must be persisted (Section 8.4), and observations that an operator discards, for example because no partner arrived within the tolerance, change the observation history and must be counted.

**Some operators are confluent.**

**Definition 3** (Confluence)**.** A topology operator is confluent if the sequence of statistical inputs it emits depends only on the order of observations within each input, and not on how the inputs interleave on arrival.

Positional pairing is confluent, as is greedy time alignment of time-ordered inputs. This is the determinacy property of Kahn networks [46]. The independent topology is not confluent: its output order is the arrival order. A key-based join that emits a row when its last component arrives is not confluent either. For a confluent topology the trajectory is independent of

scheduling; for a non-confluent one the interleaving is part of the experiment and the ordering policy has to be recorded (Section 7.1).

# 7 Execution Runtime

The runtime determines how observations reach the run. An observation may be *pushed* by a source or *pulled* by the runtime, and execution may be synchronous or asynchronous. These are properties of execution and leave (1) unchanged. In particular, a procedure does not need an asynchronous interface because its observations arrive asynchronously (Principle 1).

## 7.1 Concurrency and ordering

Several producers may deliver observations concurrently. The runtime may acquire concurrently, but the inferential state of one procedure instance has one well-defined transition order (Principle 4):

$$\text{concurrent acquisition} \longrightarrow \text{ordered delivery} \longrightarrow \text{serialized state transition.}$$

This is preferable to requiring every procedure to be thread-safe. It keeps locking out of statistical code, and it makes the order in which observations were applied a recorded fact, where otherwise it would be an accident of scheduling.

The ordering policy names the order that is intended: arrival order, per-source sequence-number order, event-time order, or an explicit ordering key. A per-source policy does not fix the interleaving *between* sources. If the topology is confluent that does not matter. If it is not, either a total order is imposed, for instance by merging on event time, or the realized order of delivery is logged so that it can be replayed (Section 8.6).

## 7.2 Batching

A runtime may deliver a batch $B = (x_{t+1}, \ldots, x_{t+k})$ in one call. Write $F^*$ for the iterated transition, that is, the fold of $F$ over a sequence of inputs that stops at a terminal state as in Definition 2. Batching is *transparent* if

$$F^*(S_t, R_t;\, B) \;=\; F^*(S_t, R_t;\, x_{t+1}, \ldots, x_{t+k}), \tag{4}$$

including all intermediate outputs. A transparent batch is a transport optimization and nothing else.

The condition is stricter than it looks for procedures with a stopping rule. If a decision is evaluated only at the end of each batch, the procedure can stop only at batch boundaries. That is a group-sequential design [55], a different statistical procedure with different error probabilities and sample-size distribution. In that case the batch structure belongs to the definition of $F$ and to the input contract, not to the runtime.

## 7.3 Duplicates and delivery guarantees

Distributed transports duplicate and reorder messages. Using source identifiers and sequence numbers, the runtime can deduplicate, hold observations back until a sequence is contiguous, or reject gaps outright. Combined with a checkpoint that records the per-source position atomically with the inferential state (Section 8.4), at-least-once delivery yields state transitions that are applied exactly once: after a restart the source is rewound to its recorded position or earlier, and anything already applied is dropped as a duplicate.

The procedure is never responsible for broker offsets or acknowledgement protocols.

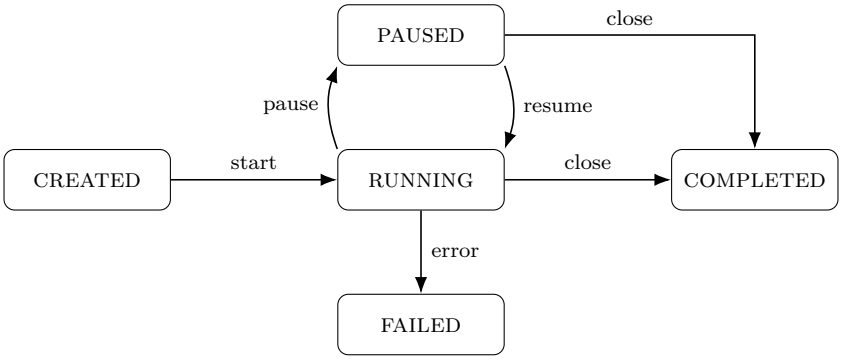



Figure 4: Run lifecycle. A run restored from a checkpoint starts in PAUSED, and attaching a runtime resumes it. A failed run is never continued in place; recovery restores an earlier checkpoint.

### 7.4 Backpressure

Inputs may arrive at very different rates. The runtime can respond by buffering, blocking producers, throttling, sampling, or dropping. Only blocking and lossless buffering are pure execution policies. Sampling and dropping change the observation history, so the policy and the number of affected observations become part of the experiment and must appear in its provenance.

## 8 Runs: Lifecycle, Failure, and Persistence

A run is one concrete execution of a procedure. A runtime is a disposable environment that drives it. An *experiment* is a higher-level scientific object that may contain several runs.

### 8.1 Initialization and restoration

A new run may take its initial state $S_0$ from the procedure's default, an explicit prior, historical data, or the final state of an earlier run. All of these *initialize* a new run: it has a new identity and an empty history. *Restoring* a checkpoint is different. It continues an existing run with its identity and history intact. Two runs may hold numerically identical states and still differ in provenance, and the architecture keeps the two operations apart for that reason.

### 8.2 Lifecycle

A procedure has a minimal lifecycle: it is active until its state is terminal. A run has a richer one (Figure 4). The run owns the execution lifecycle; the procedure owns the inferential semantics.

Reaching a terminal state of the procedure does not by itself end the run. The runtime may notify consumers, drain or discard pending data, close sources, and only then close the run. Inputs that arrive after a terminal state are not applied (Section 5.3); they are counted.

### 8.3 Errors and transactional updates

Failures occur at different boundaries and are the responsibility of different layers (Table 3).

That no failure modifies the inferential state is by design. A state update is a transaction,

$$\text{validate} \longrightarrow \text{compute} \longrightarrow \text{commit},$$

Table 3: Error classes, the layer responsible for them, and their consequence. None of them modifies the inferential state.

| **Error** | **Responsible layer** | **Consequence** |
|---|---|---|
| Source error | adapter, runtime | run stays valid |
| Delivery or ordering error | runtime, topology | run fails |
| Validation error | contract, topology | skipped and reported, or fails |
| Numerical or inference error | procedure | transition not committed; run fails |
| Serialization error | persistence | previous checkpoint stays valid |
| Consumer error | consumer | recorded; inference continues |

Table 4: Contents of a run checkpoint.

| **Component** | **Purpose** |
|---|---|
| Procedure identity | name, version, and configuration; checked on restore |
| Inferential state $S_t$ | continues the mathematics |
| Random state $R_t$ | continues a randomized procedure exactly |
| Topology state | observations consumed from sources but not yet applied |
| Source positions | where each source resumes: file offsets, cursors, broker offsets, sequence numbers |
| History digest | identifies the input history that led to $S_t$ |
| Observation log position | record count, byte offset and hash chain of the observation log, if one is kept |
| Provenance | see Section 8.5 |
| Format version | allows the checkpoint format to evolve |

and anything that fails before the commit leaves $(S_{t-1}, R_{t-1})$ in place. The simplest way to obtain this is to make states immutable values and $F$ a function that returns a new state: there is then nothing to roll back.

A consumer failure is not an inference failure (Principle 6). A visualization that crashes must not invalidate an otherwise valid trajectory.

An invalid input is not an inference failure either. A run does not control the stream it consumes, and a single malformed message must not make it unusable. The reference implementation therefore skips invalid inputs by default: an observation the topology cannot use, a statistical input that violates its contract, or one that the transition itself finds invalid. Nothing is committed, so skipping is exact. Every such input is reported as an incident, whatever the policy, and counted when it is skipped. Stopping the run on invalid input is an explicit choice, and the experimenter may name the error it raises. A separate threshold stops the run after a given number of consecutive invalid inputs: one bad message is noise, a source that sends nothing valid is broken. The policy is recorded in provenance. Contracts themselves are strict: a value of a foreign numeric type, such as a `numpy` scalar, is refused with a message that says how to convert it in the adapter, rather than interpreted.

### 8.4 Checkpoints

We distinguish a *snapshot*, which represents state for inspection, from a *checkpoint*, which is sufficient to continue the run. Table 4 lists what a run checkpoint contains. It contains no sockets, threads, event loops, or other runtime resources (Principle 5).

**Consistency.** A checkpoint must describe a single cut through the execution: every observation before a source's recorded position is either reflected in $S_t$ or held in the topology state, and no observation at or after that position is. With a single writer this is easy to guarantee. Checkpoints are taken between transitions, when all components refer to the same prefix of every source. A system that runs topology operators and the procedure in separate processes needs a distributed snapshot protocol to obtain the same cut [44, 45]. Restoration from a consistent checkpoint continues the trajectory from $S_t$ without altering the semantics of the history already processed.

**History digest.** Let $h_0$ be a fixed constant and $h_t = H(h_{t-1} \,\|\, \mathrm{enc}(X_t))$ for a cryptographic hash $H$ and a canonical encoding of statistical inputs. Two runs with equal digests applied the same input history, up to hash collisions. The digest lets a replay be verified without storing the history in the checkpoint. For the digest to be comparable across languages, the encoding must not depend on how a language prints floating-point numbers. The reference implementation uses the JSON Canonicalization Scheme [56], which fixes number formatting (the shortest representation that round-trips), string escaping and key order. Integers that a binary64 value cannot hold exactly are refused by the input contracts, and 64-bit values such as $R_t$ are stored as strings. The same requirement applies earlier, to observations: one whose value cannot be recorded exactly in JSON (a non-finite number, an integer beyond $2^{53}$, a string that UTF-8 cannot encode, a source name that is not a string) is invalid input, rejected before it reaches a topology buffer, so that neither the observation log nor a checkpoint ever holds a value that a replay would read back differently.

### 8.5 Provenance

A reproducible run records the procedure identity, version, and configuration; the input contracts; the identity of every source; the topology; the ordering and delivery policy; the runtime mode of every execution segment; the numerical backend; the seed; units and calibration where relevant; counts of everything that was skipped or dropped; and the lifecycle events with their timestamps. Provenance is part of the scientific representation of a run. It is not an operational log that can be discarded once the run has finished.

### 8.6 Replay and simulation

Because a procedure sees only logical inputs, the same input can be supplied by a live source, by historical data, by a recorded observation log, or by a simulator. Replay is then ordinary execution with a different adapter. It supports debugging, regression testing, comparison of methods on identical histories, and regeneration of figures.

An observation log is part of the run's persistent state. It records the observations delivery passes on, not every arrival: duplicates and observations still held back are left out, so that its position and the source positions describe the same cut. The checkpoint records its position, and restoring the checkpoint verifies the log against the recorded hash chain and cuts it back to that position: observations logged after the checkpoint belong to an execution that was abandoned. The run does not preserve them. After a restore they are delivered again by the sources from their recorded positions, and a source that cannot deliver past observations again must be buffered by the application, between the source and the run.

A simulator can also close the loop of an adaptive experiment,

$$\text{procedure output} \longrightarrow \text{experimental design} \longrightarrow \text{simulator} \longrightarrow \text{new observation},$$

without any change to the procedure.

# 9 Numerical Semantics

Long-running procedures accumulate many small terms. Whether a run can be reproduced, and whether two implementations agree, are therefore questions about floating-point arithmetic [57, 58] as much as about statistics. The usual mechanisms apply: likelihoods are kept in the log domain, normalizing constants are computed with a stable log-sum-exp, long sums use compensated accumulation [59, 60], and non-finite values are rejected before they are committed. The runtime enforces the last point for every procedure: a transition that overflows, or yields a non-finite state or output, raises a numerical error and commits nothing. Procedures that estimate a variance also report a numerical error when rescaled data make the estimate underflow, rather than deciding on a meaningless statistic.

More important for the architecture is to say what "equal trajectories" means. We distinguish four levels.

**Bitwise equality.**
The same implementation on the same platform performs the same operations in the same order. Checkpoint, replay, batch, and synchronous/asynchronous equivalence should hold at this level, because none of them is supposed to change which operations are performed.

**Tolerance equality.**
Different backends, compilers, or languages agree on states and continuous outputs within declared absolute and relative tolerances.

**Decision equality.**
Discrete outputs (decisions, alarms, counts) are equal. Tolerance equality does not imply decision equality: a decision is a discontinuous function of the state, and two implementations that agree to $10^{-12}$ can disagree about a threshold crossing. A numerical contract must say how such cases are treated, for example by reporting the margin to the boundary and flagging decisions taken within tolerance of it.

**Distributional equality.**
Randomized procedures driven by different generators agree only in distribution, unless the generator itself is part of the contract.

Compensation terms are part of the inferential state. A compensated sum that is restored without its compensation term continues with a different value than the uninterrupted run.

Bitwise equality is fragile in ways that have nothing to do with the statistics. Python's built-in `sum` adds floating-point numbers with compensation since version 3.12 and without it before, so a particle filter whose weights are summed with it follows a different trajectory, in the last bits, on different versions of the same language. The reference implementation found this through its conformance vectors and now sums floats on the inference path only by an explicit left-to-right loop, or by the compensated sum whose correction term is stored in the state.

The levels can be tested across implementations only if there is something to compare against. The reference implementation ships language-independent conformance vectors: for each of sixteen cases, the inputs, every output, the final state, the random state and the history digest, all encoded canonically. A second implementation conforms bitwise if it reproduces them exactly, and at the tolerance or decision level if it reproduces them within its declared contract.

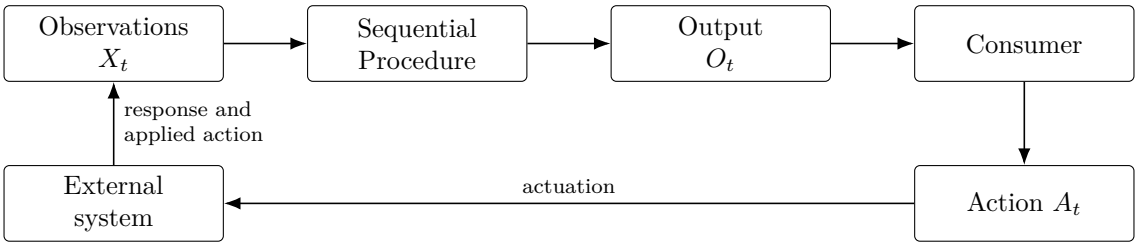



Figure 5: Inference and action are separated. The loop closes through the external system. An action that influences later observations is itself recorded.

# 10 Consumers, Adapters, and Feedback

## 10.1 Adapters

Physical and external systems are connected through adapters: imaging archives speaking DICOM [61], industrial equipment exposed via OPC UA [62], sensor networks using MQTT [63], databases, files, laboratory instruments, and network services. A medical imaging pipeline, for instance, is

$$\text{scanner} \to \text{DICOM} \to \text{adapter} \to \text{logical image input} \to \text{topology} \to \text{run}.$$

DICOM metadata may be used for routing, pairing, filtering, and provenance without becoming part of the statistical procedure. The same logical input can be rebound to another scanner, an archive, or a phantom simulator, provided the new source satisfies the input contract.

## 10.2 Consumers

A consumer receives outputs $O_t$. It has no access to the inferential state and cannot change it (Principle 6). Typical consumers are visualization, logging, reporting, database storage, alerting, experimental design, and control or actuator integration. The first four are passive. The last three act on the world.

## 10.3 Feedback

Inference is $X_{1:t} \mapsto S_t \mapsto O_t$. A consumer may implement $O_t \mapsto A_t$, where $A_t$ is an external action. If the action changes the subsequent observation process, so that $X_{t+1}$ depends on $A_t$, the system contains a feedback loop (Figure 5).

The separation lets one procedure serve passive and active experiments alike. It leaves two obligations, which should be stated plainly.

*Remark* 1 (Separation does not confer validity). Moving actuation out of the procedure does not make inference valid under feedback. Whether a procedure remains valid when the design depends on earlier outputs is a statistical property. Likelihood-ratio and e-process arguments survive designs that are predictable with respect to the past [29]; many fixed-sample arguments do not. The architecture contributes two things here. It makes the dependence explicit, and it lets the same $F$ be analyzed with and without the loop.

*Remark* 2 (Actions are data). An action that changes the observation process is part of the history of the experiment. It should either re-enter the statistical boundary as a declared logical input, as a control input does in a Kalman filter, or be recorded in provenance. A consumer never calls back into the run. The loop always closes through a source.

# 11 Composition and Stateful Predictive Models

## 11.1 Composition

Procedures compose with one another and with stateless transformations. In

$$\text{raw input} \to \text{exponential moving average} \to \text{residual} \to \text{change-point procedure}$$

the moving average is a stateful transformation and therefore itself a procedure, the residual is a stateless map, and the detector is a procedure. Feeding an e-process into a CUSUM-type detector is another composition of the same shape.

Composition is closed: the chain of two procedures is a procedure whose state is the pair of their states and whose transition applies them in order. No component inherits from a pipeline type. Composition is an external mechanism and adds no depth to any hierarchy (Principle 7).

## 11.2 Stateful predictive models

The abstraction is not restricted to classical statistics. A recurrent model with state $S_t = (h_t, c_t)$ and update $(S_t, O_t) = F(S_{t-1}, X_t)$, such as an LSTM [35] used for prediction, satisfies Definition 1. Its trained weights are configuration; they are part of the procedure's identity and do not change during a run. If the weights are updated online, they become part of the state.

A stateless mapping $O = f(X)$ is not a procedure. It is a transformation in the topology or pipeline. This distinction admits stateful predictive models without turning the architecture into a general machine-learning framework. The criterion for membership is whether a method maintains state and produces outputs sequentially. Which statistical school it belongs to is irrelevant.

# 12 Reference Procedure: The Kiefer–Weiss Problem

We use a sequential test as the reference procedure because it exercises most of the architecture at once: log-domain accumulation, a stopping boundary, a terminal decision, and state that must survive long runs.

**Problem.** Let $X_1, X_2, \ldots$ be independent with density $f_\theta$, and consider testing $\theta_0$ against $\theta_1 > \theta_0$ with error probabilities at most $\alpha_0$ and $\alpha_1$. The SPRT minimizes the expected sample size at $\theta_0$ and at $\theta_1$ [26]. Between them its expected sample size can be much larger, and its sample size is unbounded. Kiefer and Weiss [64] asked for the test that minimizes the *maximum* expected sample size $\sup_\theta \mathbb{E}_\theta N$ under the same error constraints. The *modified* Kiefer–Weiss problem minimizes $\mathbb{E}_{\theta^*} N$ at one intermediate point $\theta^*$; in symmetric normal and binomial problems the two coincide [65].

**The 2-SPRT.** Lorden [66] proposed running two one-sided SPRTs against the intermediate point. With

$$L_t^0 = \sum_{i=1}^{t} \log \frac{f_{\theta^*}(X_i)}{f_{\theta_0}(X_i)}, \qquad L_t^1 = \sum_{i=1}^{t} \log \frac{f_{\theta^*}(X_i)}{f_{\theta_1}(X_i)}, \tag{5}$$

the test stops at $N = \inf\{t : L_t^0 \geq a_0 \text{ or } L_t^1 \geq a_1\}$ and rejects $\theta_0$ if the first boundary was reached and $\theta_1$ if the second was. Lorden showed that the 2-SPRT approximately minimizes $\mathbb{E}_{\theta^*} N$ among tests with the same error probabilities, and Huffman [67] gave an explicit choice of $\theta^*$ for which it is an asymptotically efficient solution of the original minimax problem.

Under $\theta_i$ the process $\exp(L_t^i)$ is a non-negative martingale with unit mean, so by Ville's inequality [68]

$$\mathbb{P}_{\theta_0}(\text{reject } \theta_0) \le e^{-a_0}, \qquad \mathbb{P}_{\theta_1}(\text{reject } \theta_1) \le e^{-a_1}. \tag{6}$$

The thresholds $a_i = \log(1/\alpha_i)$ therefore guarantee the error rates. They are conservative, because they ignore that the other boundary may be reached first.

**Choice of $\theta^*$.** To first order the two boundaries should be reached at the same time under $\theta^*$. With $I(\theta, \theta')$ the Kullback–Leibler divergence, this gives

$$\frac{a_0}{I(\theta^*, \theta_0)} = \frac{a_1}{I(\theta^*, \theta_1)}. \tag{7}$$

For normal observations with known variance $\sigma^2$ the solution is $\theta^* = \theta_0 + (\theta_1 - \theta_0)\sqrt{a_0}/(\sqrt{a_0} + \sqrt{a_1})$, the midpoint when $a_0 = a_1$. In this case the continuation region is a triangle in the $(t, \sum_i X_i)$ plane, and the test always stops by

$$n_{\max} = \left\lceil \frac{2\sigma^2\,(a_0/\delta_0 + a_1/\delta_1)}{\delta_0 + \delta_1} \right\rceil, \qquad \delta_i = |\theta^* - \theta_i|. \tag{8}$$

**As a Sequential Procedure.** The state is $S_t = (t, L_t^0, L_t^1, d_t)$, with each sum carried together with its compensation term and $d_t$ the decision, if any. The terminal set is $\{d_t \neq \varnothing\}$. The output repeats the statistics and the decision. The procedure owns the meaning of this state and its stopping semantics. It owns no broker integration, scheduling, acquisition, checkpoint storage, visualization, or reporting.

**Executing optimal plans.** The 2-SPRT is an approximation. Exact solutions of the Kiefer–Weiss problem can be computed numerically by backward induction for Bernoulli observations, discrete exponential families, and normal observations [22, 23, 24]. The result of such a computation is a sampling plan: a horizon $H$ and, for every step $n \le H$, an interval of the running sum $\sum_{i \le n} X_i$ in which sampling continues. Design and execution are different problems. The design is computed once, offline. Its execution on a stream is a Sequential Procedure with state $(n, \sum_{i \le n} X_i, d_n)$, and the reference implementation provides one that takes any plan of this form as its configuration. The plan thereby becomes part of the procedure's identity and is recorded in provenance. As a check, the plan formed by the two boundary lines of the Gaussian 2-SPRT reproduced the stopping time and decision of the 2-SPRT on all 4 000 simulated paths we compared.

The reference implementation also contains the design side, following [22, 23, 24]: backward induction on the Lagrangian $E_{\theta^*} N + \lambda_0 P_{\theta_0}(\text{reject}) + \lambda_1 P_{\theta_1}(\text{accept})$ over plans with horizon $H$ [69], with the multipliers adjusted until the error probabilities meet their targets. The running sum is carried on a lattice: the integers for Bernoulli and Poisson data, where every quantity is exact, a lattice of step $h$ for normal data, and the midpoints of cells of width $h$ for exponential data, where the error is of order $h^2$. Writing the error probabilities as expectations under $\theta^*$ of the likelihood ratios lets one backward pass return the Lagrangian and both error probabilities at once, so that adjusting the multipliers by Broyden's method takes a few passes. The resulting plan is executed by the same plan executor. For $\theta_0 = 0.3$ against $\theta_1 = 0.5$ with $\alpha_0 = \alpha_1 = 0.05$ and $H = 150$, it attains error probabilities 0.0498 and 0.0496 and a maximum expected sample size of 45.2, against 50.8 for Wald's SPRT with thresholds calibrated to the same error probabilities (56.9 with Wald's approximate thresholds $\pm \log 19$). The value of the Lagrangian obtained by backward induction agrees with the forward evaluation of the plan to machine precision, and the exact operating characteristic agrees with simulation through the executor. For a normal mean, $\theta_0 = 0$ against $\theta_1 = 0.5$ with $\sigma = 1$ and $H = 120$, the plan

computed on a lattice of step $\sigma/16$ has error probabilities $0.0515 \pm 0.0015$ when executed on simulated normal data, and an expected sample size at $\theta^*$ of 31.0. The 2-SPRT with thresholds calibrated to the same error probabilities ($a = 2.16$) needs 31.3: in this example Lorden's approximation is within about one percent of the optimum, which the backward induction makes it possible to quantify.

The modified problem fixes $\theta^*$. The Kiefer–Weiss problem asks for the $\theta^*$ at which the resulting test has the smallest maximum expected sample size, and by Lorden's characterisation [69] that is the $\theta^*$ at which the maximum is attained at $\theta^*$ itself. For discrete data the implementation generates candidates by bisection on $g(\theta^*) = \arg\max_\theta E_\theta N - \theta^*$, designing each in full, and returns the candidate, the first-order point included, with the smallest maximum expected sample size. For the Bernoulli example the first-order $\theta^* = 0.3971$ gives a maximum expected sample size of 45.17, attained at 0.3933; the search returns $\theta^* = 0.3925$ with 45.01, attained at 0.3936. Near the solution the maximum is flat in $\theta^*$, and for discrete data it moves in small steps because the attainable error probabilities do, so the gain over the first-order point is small and the characterisation holds only approximately.

For normal data the characterisation itself is well determined: the function $g(\theta^*) = \arg\max_\theta E_\theta N - \theta^*$ is smooth and decreasing, and on a symmetric problem its root is the midpoint to $3 \cdot 10^{-4}$ of $\theta_1 - \theta_0$. The maximum expected sample size is not: over a tenth of the interval around the root it changes by about $2 \cdot 10^{-4}$ of its value, the order of the calibration error of the multipliers, so that which candidate has the smallest maximum would be partly decided by rounding, and would change with the candidates a search happens to visit. For $\theta_0 = 0$ against $\theta_1 = 0.5$ with error probabilities 0.05 and 0.2 and $H = 60$, the candidate with the smallest maximum among those of a bisection over the whole interval is $\theta^* = 0.340$ with 17.131, the root of the characterisation is 0.3462 with 17.140, and the first-order point 0.289 gives 17.148: the three plans differ by less than a tenth of a percent. For data on a lattice (normal, exponential) the implementation therefore returns the root of the characterisation, found by the Illinois variant of regula falsi to $2 \cdot 10^{-4}$ of $\theta_1 - \theta_0$; it is reproducible to that tolerance, and a symmetric problem gets the midpoint to it.

Because the root is well determined, it can be tabulated. In the standardised coordinate $\lambda = (\theta^* - \theta_0)/(\theta_1 - \theta_0)$ it depends on $\alpha_0$, $\alpha_1$ and the horizon, which enters through $\rho = \delta\sqrt{H}/(z_{\alpha_0} + z_{\alpha_1})$, the ratio of the horizon to the shortest fixed-sample test; exchanging the hypotheses maps $\lambda$ to $1 - \lambda$, so the correction to the first-order point is antisymmetric in $(\alpha_0, \alpha_1)$. A pilot study showed it smooth in $\log\alpha_0$, $\log\alpha_1$, nearly constant in $\rho$ above 1.5 (and in $\delta$ at fixed $\rho$, to $1.5 \cdot 10^{-3}$), with a boundary layer of up to 0.015 as $\rho$ approaches 1. The roots were computed on Gauss–Lobatto–Legendre nodes, which are D-optimal for polynomial regression [70], in tensor product (63 nodes for $\alpha_0, \alpha_1 \in [0.005, 0.2]$, $\rho \in [1.5, 2.5]$, only on one side of the diagonal), and fitted by an antisymmetric Legendre series of degree 5 in each $\log\alpha$ and 1 in $\rho$; on 11 Halton points not used in the fit the error is $1.2 \cdot 10^{-4}$ rms and at most $3.3 \cdot 10^{-4}$. The search for the least favourable $\theta^*$ starts from a bracket of $\pm 0.004$ around the tabulated root ($\pm 0.03$ below $\rho = 1.5$) and falls back to the whole interval when the bracket does not contain a sign change. The example above takes 83 seconds; the bisection over the whole interval with the earlier implementation, without numpy and without the calibration below, took two hours.

Each candidate is designed in full. The backward pass is written as array operations (numpy, when installed), and once Broyden's method has come within $10^{-3}$ of the targets the multipliers are adjusted by a small joint raise before the slower coordinate search is tried; on thirteen designs across the four families both changes leave the plans unchanged.

For discrete data there is a further limit. The plans that are optimal for some pair of multipliers are non-randomised, and near the smallest feasible horizon they need not reach every pair of error probabilities that some test of that size reaches; randomised boundaries would. The implementation then tries smaller horizons, which also respect the constraint, and

reports a failure only if none succeeds. Before any of this it compares the targets with the Neyman–Pearson bound: a plan that stops by $H$ is a test based on $H$ observations, so its $\alpha_1$ is at least that of the most powerful randomised level-$\alpha_0$ test of that size, computed exactly for Bernoulli, Poisson and normal data. Targets below the bound are reported at once, instead of after a search that cannot succeed.

**Validation.** A reference procedure can be validated against analytical facts without any other implementation. For the 2-SPRT these are the error bound (6), the closed-form $\theta^*$, and the sample-size bound (8), which constant data equal to $\theta^*$ attains exactly. Table 5 adds a Monte Carlo study for $\theta_0 = 0$, $\theta_1 = 0.5$, $\sigma = 1$, and nominal error rates 0.05. The SPRT with Wald's thresholds attains error rates of about 0.038. The 2-SPRT with the guaranteed thresholds is more conservative and, as a result, slower. Once its threshold is calibrated to the SPRT's attained error rate, the trade-off the Kiefer–Weiss problem is about becomes visible. The expected sample size at the midpoint drops from 42.4 to 37.1, the expected sample sizes at the hypotheses rise from about 24.3 to about 27.0, and the largest sample observed falls from 384 to 77, within the bound $n_{\max} = 79$.

Table 5: Operating characteristics for normal observations with unit variance, $\theta_0 = 0$ against $\theta_1 = 0.5$; 20 000 replications per cell, standard errors in parentheses. All three tests see the same simulated data at each $\theta$. The matched threshold was calibrated on a separate random stream.

| **Test** | $\theta$ | $\mathbb{E}_\theta N$ | $\max N$ | $\mathbb{P}_\theta(\text{reject } \theta_0)$ |
|---|---|---|---|---|
| SPRT, Wald thresholds $\pm 2.944$ | 0 | 24.21 (0.12) | 177 | 0.0379 |
| | 0.25 | 42.36 (0.24) | 384 | 0.5029 |
| | 0.5 | 24.44 (0.12) | 182 | 0.9616 |
| 2-SPRT, $a = \log 20 \approx 2.996$ | 0 | 32.96 (0.10) | 91 | 0.0218 |
| ($n_{\max} = 96$) | 0.25 | 48.15 (0.12) | 93 | 0.5041 |
| | 0.5 | 33.15 (0.10) | 93 | 0.9788 |
| 2-SPRT, matched $a = 2.456$ | 0 | 26.90 (0.08) | 75 | 0.0355 |
| ($n_{\max} = 79$) | 0.25 | 37.09 (0.10) | 77 | 0.5030 |
| | 0.5 | 27.05 (0.08) | 72 | 0.9634 |

The study drives the procedures with a bare loop over simulated values. It uses no run, runtime, or topology. That the same transition function serves a Monte Carlo study and a laboratory deployment unchanged is the practical content of Principle 1.

## 13 Conformance Invariants

An architecture of this kind is not evaluated well by throughput benchmarks. Its claims are about semantics, and they can be stated as invariants that an implementation either satisfies or does not. Each invariant below compares trajectories in the sense of Definition 2, at the bitwise level of Section 9 unless noted otherwise.

**Invariant 1** (Synchronous/asynchronous equivalence)**.** For the same ordered statistical input history, synchronous and asynchronous execution yield the same trajectory. For a confluent topology the same holds for any interleaving of the sources.

**Invariant 2** (Checkpoint equivalence)**.** For every $k$, processing $x_{1:k}$, taking a checkpoint, restoring it from storage with no other state carried over, and processing $x_{k+1:T}$ yields the same

Table 6: Invariants and the principles they test.

| Invariant | Principles | What a violation indicates |
|---|---|---|
| 1 Sync/async | 1, 4 | scheduling leaks into inference |
| 2 Checkpoint | 5 | state hidden outside the checkpoint |
| 3 Replay | 3, 5 | unrecorded inputs or randomness |
| 4 Batch | 1 | transport changes the statistics |
| 5 Consumers | 6 | outputs feed back into state |
| 6 Runtime | 1, 3 | the procedure depends on a source |
| 7 Topology | 2 | pairing logic inside the procedure |

trajectory as processing $x_{1:T}$ without interruption. The same holds if the sources redeliver observations from before the checkpoint.

**Invariant 3** (Replay equivalence)**.** Replaying the recorded delivered history of a run through the same delivery policy and topology reproduces its trajectory and its history digest.

**Invariant 4** (Batch transparency)**.** Delivering a history in batches of any size yields the same trajectory as delivering it element by element, including the step at which a terminal state is reached.

**Invariant 5** (Consumer non-interference)**.** Attaching, detaching, or crashing consumers does not change the trajectory.

**Invariant 6** (Runtime independence)**.** Different acquisition mechanisms that produce the same ordered logical input history yield the same trajectory.

**Invariant 7** (Topology semantics)**.** Each topology operator reproduces known reference histories, and operators declared confluent emit the same sequence for every interleaving of their inputs.

Table 6 relates the invariants to the principles. The invariants are stated for a single implementation. Agreement *between* implementations in different languages is a separate validation criterion, at the tolerance and decision levels.

## 14 Reference Implementation

The architecture is defined semantically (Principle 7). An implementation must provide equivalent contracts for state transition, input validation, output generation, lifecycle, persistence, ordering, and reproducibility. It need not use classes or inheritance. A language with structures and functions, traits, or type classes can express the same contracts, and implementations in different languages need not have the same feature set.

We provide a reference implementation in Python. It uses only the standard library (numpy, when installed, accelerates the design of optimal plans) and comprises about 6 600 lines, with one module per layer of Figure 1. The contract a procedure has to satisfy is small:

```
config()                -> parameters that determine F
inputs()                -> {name: input contract}
initial_state()         -> S_0
step(state, x, rng)     -> (new state, output)
is_terminal(state)      -> bool
encode_state(state), decode_state(value)
```

States are immutable values and `step` is a pure function of its arguments. The run draws the statistical random state into a fresh generator before each step and commits it together with the new state, which gives transactional updates without rollback logic. The random state is a single 64-bit integer [71], so it is trivial to checkpoint and to reproduce in another language. A checkpoint is a plain JSON value.

The implementation includes fourteen procedures. For testing simple hypotheses: Wald's SPRT, the 2-SPRT of Section 12, and an executor for precomputed sampling plans. For clinical trials: a group sequential test with efficacy and non-binding futility boundaries, for known or estimated variance. For anytime-valid inference: the mixture SPRT for a normal mean [72, 10], its counterpart for unknown variance, whose Bayes factor with a scale-invariant prior on $\sigma$ is a test martingale for every $\sigma$ [73, 74], both with an always-valid $p$-value and a confidence sequence, and two nonparametric tests by betting [75, 76, 29], one for the mean of bounded observations and a sign test of $P(X > m) = P(X < m)$, both valid for every distribution satisfying the null hypothesis. The sign test is a test of the median only when the distribution has no atom at $m$: with ties at $m$, the median can be $m$ while the signs are unbalanced, and the test then rejects. For change-point detection: CUSUM [3] and the Shiryaev–Roberts procedure [77, 78, 79]. For filtering and estimation: a Kalman filter for the local-level model with an optional control input, a bootstrap particle filter for the same model, a two-sample mean difference that accepts unpaired inputs, and an exponential moving average. Likelihood-ratio procedures take one of four families (normal with known variance, Bernoulli, Poisson, exponential). A design module computes operating characteristics and optimal plans for Bernoulli, Poisson, normal and exponential data, for a fixed or the least favourable $\theta^*$ (Section 12). Group sequential boundaries are computed from alpha- and beta-spending functions [80, 55] by recursive numerical integration [81]; the O'Brien–Fleming-type efficacy boundaries for three and five analyses reproduce the published values to three decimals, and with estimated variance each boundary is converted to a $t$ boundary of the same significance level. The $t$ distribution is computed from the standard library alone: the incomplete beta function by its continued fraction, in logarithms where the tail underflows, and Fisher's expansion to third order when the degrees of freedom are large; on a grid of degrees of freedom from 0.1 to $10^{200}$ and arguments up to $10^{300}$ it agrees with 80-digit reference values to a relative error below $10^{-10}$. The implementation also includes four topology operators (independent, positional pairing, key join, time alignment), three delivery policies, a synchronous pull runtime, and an asynchronous push runtime with blocking or dropping backpressure. Transformations applied by a topology or between the stages of a chain carry a name, a version and a configuration, like procedures, and that identity is part of the checkpoint. A checkpoint is validated field by field before any of it is trusted, so that a damaged or foreign one is reported as such.

The invariants of Section 13 are executable tests. Checkpoint equivalence, batch transparency, and consumer non-interference are checked for fifteen combinations of procedure and topology, including the randomized particle filter and topologies whose buffers are non-empty at the checkpoint. The remaining invariants are checked on representative cases. All seven hold bitwise. The suite has 267 tests in total; the remainder cover transactional updates, lifecycle transitions, delivery policies, invalid-input policies, compatibility checks on restore, the observation log, and the statistical properties claimed for each procedure: error rates under continuous monitoring, coverage of confidence sequences, run lengths to false alarm, optimality of designed plans, Lorden's characterisation of the least favourable $\theta^*$, the alpha and beta spent by group sequential designs, and the power of designs with futility boundaries. Table 5 is produced by a script in the repository from a fixed seed.

An independent implementation of the history digest and of the SPRT in JavaScript reproduces the conformance vectors bit for bit: all sixteen digests, the canonical encodings of 500 numbers, and every SPRT output for normal, Bernoulli and Poisson data.

Two examples show what the separation buys. A paired test fed by two concurrent

producers with random delays produces the same history digest as the synchronous run on the same data, with no asynchronous code in the procedure. A closed-loop example uses a controller that is a consumer and acts on a simulated plant. The applied action returns as a logical input paired with the measurement it influenced, and the filter remains an ordinary Kalman filter with a control term.

# 15 Discussion

**What the separation provides.** A procedure becomes reusable because its definition is no longer tied to acquisition infrastructure. A topology becomes explicit because pairing and alignment are represented independently of the statistical update. A run becomes reproducible because inferential state, random state, topology state, source positions, and provenance are persisted together and nothing else is needed to continue. Visualization and experimental design obtain a natural boundary: neither has to live inside the inference code.

**What it costs.** The architecture asks more of a method author in one respect. The state must be explicit and serializable, and all randomness must pass through $R_t$. Methods written as loops over arrays have to be rewritten as transitions. This is a small change for recursive statistics and a larger one for methods that revisit past data. Such methods must carry the data they revisit in their state, at the corresponding cost in checkpoint size.

**Limitations.** The reference implementation covers fourteen procedures; optimal design is limited to simple hypotheses about one-parameter families, and continuous data are designed on a lattice. It demonstrates the contract and the conformance tests; it is not yet the general-purpose library that the gap described in Section 1 calls for. It is a single process and has not been evaluated for throughput. Its adapters read CSV files, simulate, and replay recorded observation logs; it contains none for instruments or message brokers. Cross-language agreement is demonstrated bitwise for the history digest and for the SPRT, but not yet for a complete second implementation, nor at the tolerance and decision levels for procedures that use transcendental functions, whose last bits may differ between mathematical libraries. Finally, the architecture does not address the statistical validity of adaptive experiments (Section 10.3). It makes the adaptation visible, and establishing validity remains the job of the statistical analysis.

**Scope.** The architecture does not attempt to solve distributed stream processing, machine learning, laboratory automation, or control. Those systems integrate with it through the boundaries described above. The restriction is intentional: the objective is a stable semantic boundary around sequential scientific inference, small enough to be implemented and verified.

# 16 Conclusion

We proposed an architecture for sequential scientific analysis in which statistical inference is separated from acquisition, topology, execution, persistence, and external action. The central abstraction is the Sequential Procedure,

$$(S_t,\, O_t,\, R_t) \;=\; F(S_{t-1},\, X_t,\, R_{t-1}),$$

where $X_t$ may carry any non-empty subset of several logically distinct inputs. Data topology determines how inputs are related. The runtime determines how they are acquired and ordered. A run owns state, lifecycle, persistence, and provenance. Adapters bind logical inputs to physical systems. Consumers interpret outputs and may act on the world:

**Procedure infers; consumers act.**

The separation is testable. Seven invariants follow from the principles, and a reference implementation satisfies all of them bitwise for procedures ranging from the 2-SPRT for the Kiefer–Weiss problem to a randomized particle filter. The architectural claim is not tied to a statistical method or a programming language: the state transition of a sequential scientific procedure should remain independent of the mechanisms that acquire observations, execute computation, persist state, and act on outputs. That independence is what makes sequential experiments reproducible, composable, and long-lived.

## Code availability

The reference implementation is released as the Python package `sequential-analysis`, version 0.1.1, available from the Python Package Index (`https://pypi.org/project/sequential-analysis/`) and at `https://github.com/andre-a-no/sequential-analysis`, together with the conformance vectors, the JavaScript check and the scripts that produce Table 5 and the table of least favourable parameters. It is distributed under the GNU Affero General Public License, version 3, or under a commercial license from Novikov Laboratories LLC.

## References


[1] Abraham Wald. "Sequential Tests of Statistical Hypotheses". In: *The Annals of Mathematical Statistics* 16.2 (1945), pp. 117–186.

[2] Abraham Wald. *Sequential Analysis*. New York: John Wiley & Sons, 1947.

[3] E. S. Page. "Continuous Inspection Schemes". In: *Biometrika* 41.1–2 (1954), pp. 100–115.

[4] H. Vincent Poor and Olympia Hadjiliadis. *Quickest Detection*. Cambridge University Press, 2009.

[5] Rudolph E. Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45.

[6] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.

[7] Lars Buitinck et al. "API Design for Machine Learning Software: Experiences from the scikit-learn Project". In: *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 2013, pp. 108–122.

[8] hmmlearn developers. *hmmlearn: Hidden Markov Models in Python, with scikit-learn like API*. URL: `https://hmmlearn.readthedocs.io`.

[9] Ramesh Johari et al. "Peeking at A/B Tests: Why It Matters, and What to Do About It". In: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2017, pp. 1517–1525. DOI: `10.1145/3097983.3097992`.

[10] Ramesh Johari et al. "Always Valid Inference: Continuous Monitoring of A/B Tests". In: *Operations Research* 70.3 (2022), pp. 1806–1821. DOI: `10.1287/opre.2021.2135`.

[11] Zhenyu Zhao, Mandie Liu, and Anirban Deb. *Safely and Quickly Deploying New Features with a Staged Rollout Framework Using Sequential Test and Adaptive Experimental Design*. arXiv:1905.10493. 2019.

[12] Michael Lindon and Alan Malek. "Anytime-Valid Inference for Multinomial Count Data". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.

[13] *SAS/STAT 15.1 User's Guide: The SEQDESIGN Procedure; The SEQTEST Procedure*. SAS Institute Inc. Cary, NC, 2018.

[14] Gernot Wassmer and Friedrich Pahlke. *rpact: Confirmatory Adaptive Clinical Trial Design and Analysis*. R package. URL: https://www.rpact.org.

[15] Gernot Wassmer and Werner Brannath. *Group Sequential and Confirmatory Adaptive Designs in Clinical Trials*. 2nd ed. Springer, 2025. DOI: 10.1007/978-3-031-89669-9.

[16] Martin Schnuerch and Edgar Erdfelder. "Controlling Decision Errors with Minimal Costs: The Sequential Probability Ratio $t$ Test". In: *Psychological Methods* 25.2 (2020), pp. 206–226. DOI: 10.1037/met0000234.

[17] Sandipan Pramanik, Valen E. Johnson, and Anirban Bhattacharya. "A Modified Sequential Probability Ratio Test". In: *Journal of Mathematical Psychology* 101 (2021), p. 102505.

[18] Peter Grünwald, Rianne de Heide, and Wouter Koolen. "Safe Testing". In: *Journal of the Royal Statistical Society: Series B* 86.5 (2024), pp. 1091–1128. DOI: 10.1093/jrsssb/qkae011.

[19] Stack Overflow. *2025 Developer Survey: Technology*. 2025. URL: https://survey.stackoverflow.co/2025/technology.

[20] G. Romano et al. *focus and focus-cpt: Fast Online Changepoint Detection in R and Python*. arXiv:2607.19961. 2026.

[21] Jacob Montiel et al. "River: Machine Learning for Streaming Data in Python". In: *Journal of Machine Learning Research* 22.110 (2021), pp. 1–8.

[22] Andrey Novikov, Andrei Novikov, and Fahil Farkhshatov. "A Computational Approach to the Kiefer–Weiss Problem for Sampling from a Bernoulli Population". In: *Sequential Analysis* 41.2 (2022), pp. 198–219. DOI: 10.1080/07474946.2022.2070212.

[23] Andrey Novikov and Fahil Farkhshatov. "Design and Performance Evaluation in Kiefer–Weiss Problems When Sampling from Discrete Exponential Families". In: *Sequential Analysis* 41.4 (2022), pp. 417–434. DOI: 10.1080/07474946.2022.2109673.

[24] Andrey Novikov, Andrei Novikov, and Fahil Farkhshatov. "Numerical Solution of Kiefer–Weiss Problems When Sampling from Continuous Exponential Families". In: *Sequential Analysis* 42.2 (2023), pp. 189–209. DOI: 10.1080/07474946.2023.2193602.

[25] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[26] Abraham Wald and Jacob Wolfowitz. "Optimum Character of the Sequential Probability Ratio Test". In: *The Annals of Mathematical Statistics* 19.3 (1948), pp. 326–339.

[27] David Siegmund. *Sequential Analysis: Tests and Confidence Intervals*. New York: Springer, 1985.

[28] Alexander Tartakovsky, Igor Nikiforov, and Michèle Basseville. *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. Boca Raton: CRC Press, 2014.

[29] Aaditya Ramdas et al. "Game-Theoretic Statistics and Safe Anytime-Valid Inference". In: *Statistical Science* 38.4 (2023), pp. 576–601.

[30] Steven R. Howard et al. "Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences". In: *The Annals of Statistics* 49.2 (2021), pp. 1055–1080.

[31] Lawrence R. Rabiner. "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition". In: *Proceedings of the IEEE* 77.2 (1989), pp. 257–286.

[32] Arnaud Doucet, Nando de Freitas, and Neil Gordon, eds. *Sequential Monte Carlo Methods in Practice*. New York: Springer, 2001.

[33] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[34] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

[35] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[36] Spotify. *Confidence: Python Library for A/B Test Analysis*. URL: `https://github.com/spotify/confidence`.

[37] Apache Software Foundation. *Apache Spark MLlib: Basic Statistics (RDD-based API), Streaming Significance Testing*. URL: `https://spark.apache.org/docs/latest/mllib-statistics.html`.

[38] Albert Bifet et al. "MOA: Massive Online Analysis". In: *Journal of Machine Learning Research* 11 (2010), pp. 1601–1604.

[39] Josh Day and Hua Zhou. "OnlineStats.jl: A Julia Package for Statistics on Data Streams". In: *Journal of Open Source Software* 5.46 (2020), p. 1816. DOI: `10.21105/joss.01816`.

[40] Tyler Akidau et al. "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing". In: *Proceedings of the VLDB Endowment* 8.12 (2015), pp. 1792–1803.

[41] Paris Carbone et al. "Apache Flink: Stream and Batch Processing in a Single Engine". In: *IEEE Data Engineering Bulletin* 38.4 (2015), pp. 28–38.

[42] Paris Carbone et al. "State Management in Apache Flink: Consistent Stateful Distributed Stream Processing". In: *Proceedings of the VLDB Endowment* 10.12 (2017), pp. 1718–1729. DOI: `10.14778/3137765.3137777`.

[43] Jay Kreps, Neha Narkhede, and Jun Rao. "Kafka: A Distributed Messaging System for Log Processing". In: *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*. 2011.

[44] K. Mani Chandy and Leslie Lamport. "Distributed Snapshots: Determining Global States of Distributed Systems". In: *ACM Transactions on Computer Systems* 3.1 (1985), pp. 63–75.

[45] E. N. Elnozahy et al. "A Survey of Rollback-Recovery Protocols in Message-Passing Systems". In: *ACM Computing Surveys* 34.3 (2002), pp. 375–408.

[46] Gilles Kahn. "The Semantics of a Simple Language for Parallel Programming". In: *Information Processing 74: Proceedings of the IFIP Congress*. North-Holland, 1974, pp. 471–475.

[47] Oscar Boykin et al. "Summingbird: A Framework for Integrating Batch and Online MapReduce Computations". In: *Proceedings of the VLDB Endowment* 7.13 (2014), pp. 1441–1451.

[48] Daniel Allan et al. "Bluesky's Ahead: A Multi-Facility Collaboration for an a la Carte Software Project for Data Acquisition and Management". In: *Synchrotron Radiation News* 32.3 (2019), pp. 19–22. DOI: `10.1080/08940886.2019.1608121`.

[49] Geir Kjetil Sandve et al. "Ten Simple Rules for Reproducible Computational Research". In: *PLoS Computational Biology* 9.10 (2013), e1003285.

[50] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. "The FAIR Guiding Principles for Scientific Data Management and Stewardship". In: *Scientific Data* 3 (2016), p. 160018.

[51] Luc Moreau and Paolo Missier. *PROV-DM: The PROV Data Model*. W3C Recommendation. 2013. URL: https://www.w3.org/TR/prov-dm/.

[52] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules". In: *Communications of the ACM* 15.12 (1972), pp. 1053–1058.

[53] Alistair Cockburn. *Hexagonal Architecture*. 2005. URL: https://alistair.cockburn.us/hexagonal-architecture/.

[54] George H. Mealy. "A Method for Synthesizing Sequential Circuits". In: *The Bell System Technical Journal* 34.5 (1955), pp. 1045–1079.

[55] Christopher Jennison and Bruce W. Turnbull. *Group Sequential Methods with Applications to Clinical Trials*. Boca Raton: Chapman & Hall/CRC, 2000.

[56] Anders Rundgren, Bret Jordan, and Samuel Erdtman. *JSON Canonicalization Scheme (JCS)*. RFC 8785. 2020. DOI: 10.17487/RFC8785.

[57] David Goldberg. "What Every Computer Scientist Should Know About Floating-Point Arithmetic". In: *ACM Computing Surveys* 23.1 (1991), pp. 5–48.

[58] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2nd ed. Philadelphia: SIAM, 2002.

[59] William Kahan. "Further Remarks on Reducing Truncation Errors". In: *Communications of the ACM* 8.1 (1965), p. 40.

[60] Arnold Neumaier. "Rundungsfehleranalyse einiger Verfahren zur Summation endlicher Summen". In: *Zeitschrift für Angewandte Mathematik und Mechanik* 54.1 (1974), pp. 39–51.

[61] National Electrical Manufacturers Association. *Digital Imaging and Communications in Medicine (DICOM) Standard, NEMA PS3 / ISO 12052*. NEMA, Rosslyn, VA. URL: https://www.dicomstandard.org/.

[62] International Electrotechnical Commission. *IEC 62541: OPC Unified Architecture*. IEC, Geneva.

[63] OASIS. *MQTT Version 5.0*. OASIS Standard. 2019. URL: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[64] Jack Kiefer and Lionel Weiss. "Some Properties of Generalized Sequential Probability Ratio Tests". In: *The Annals of Mathematical Statistics* 28.1 (1957), pp. 57–74.

[65] Lionel Weiss. "On Sequential Tests Which Minimize the Maximum Expected Sample Size". In: *Journal of the American Statistical Association* 57.299 (1962), pp. 551–566.

[66] Gary Lorden. "2-SPRT's and the Modified Kiefer–Weiss Problem of Minimizing an Expected Sample Size". In: *The Annals of Statistics* 4.2 (1976), pp. 281–291.

[67] Michael D. Huffman. "An Efficient Approximate Solution to the Kiefer–Weiss Problem". In: *The Annals of Statistics* 11.1 (1983), pp. 306–316.

[68] Jean Ville. *Étude critique de la notion de collectif*. Paris: Gauthier-Villars, 1939.

[69] Gary Lorden. "Structure of Sequential Tests Minimizing an Expected Sample Size". In: *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete* 51 (1980), pp. 291–302.

[70] P. G. Guest. "The Spacing of Observations in Polynomial Regression". In: *The Annals of Mathematical Statistics* 29.1 (1958), pp. 294–299. DOI: 10.1214/aoms/1177706729.

[71] Guy L. Steele Jr., Doug Lea, and Christine H. Flood. "Fast Splittable Pseudorandom Number Generators". In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*. 2014, pp. 453–472.

[72] Herbert Robbins. “Statistical Methods Related to the Law of the Iterated Logarithm”. In: *The Annals of Mathematical Statistics* 41.5 (1970), pp. 1397–1409.

[73] Tze Leung Lai. “On Confidence Sequences”. In: *The Annals of Statistics* 4.2 (1976), pp. 265–280.

[74] Muriel Felipe Pérez-Ortiz et al. “E-Statistics, Group Invariance and Anytime-Valid Testing”. In: *The Annals of Statistics* 52.4 (2024), pp. 1410–1432.

[75] Glenn Shafer. “Testing by Betting: A Strategy for Statistical and Scientific Communication”. In: *Journal of the Royal Statistical Society: Series A* 184.2 (2021), pp. 407–431.

[76] Ian Waudby-Smith and Aaditya Ramdas. “Estimating Means of Bounded Random Variables by Betting”. In: *Journal of the Royal Statistical Society: Series B* 86.1 (2024), pp. 1–27.

[77] Albert N. Shiryaev. “On Optimum Methods in Quickest Detection Problems”. In: *Theory of Probability and Its Applications* 8.1 (1963), pp. 22–46.

[78] S. W. Roberts. “A Comparison of Some Control Chart Procedures”. In: *Technometrics* 8.3 (1966), pp. 411–430.

[79] Moshe Pollak. “Optimal Detection of a Change in Distribution”. In: *The Annals of Statistics* 13.1 (1985), pp. 206–227.

[80] K. K. Gordon Lan and David L. DeMets. “Discrete Sequential Boundaries for Clinical Trials”. In: *Biometrika* 70.3 (1983), pp. 659–663.

[81] P. Armitage, C. K. McPherson, and B. C. Rowe. “Repeated Significance Tests on Accumulating Data”. In: *Journal of the Royal Statistical Society: Series A* 132.2 (1969), pp. 235–244.